

Introduction to Object-Oriented Programming (OOP)

Object-Oriented Programming (OOP) is a method of writing programs using **objects** and **classes**. It helps make code more organized, reusable, and easier to understand.

Why use OOP?

- Organize code into **real-world entities**
- Make code **modular and reusable**
- Improve **data security and structure**
- Reduce code duplication and improve maintainability

Key Concepts in Python OOP

1. **Class:** A blueprint to create objects. It defines properties (variables) and methods (functions).
2. **Object:** An instance of a class. It represents a real-world entity.
3. **Encapsulation:** Hiding internal details and only exposing what is necessary.
4. **Abstraction:** Showing only essential features and hiding complex details.
5. **Inheritance:** One class can use properties and methods of another class.
6. **Polymorphism:** Same function name behaves differently for different classes.

Simple Example

```
class Car:
    name="Neeraj"
    game="Card Game"
    def start(self):
        print("Car started")

my_car = Car()
print(my_car.name)
my_car.start()
```

In the example above:

- `Car` is a **class**
- `my_car` is an **object**
- `start()` is a **method**

What is a Constructor?

- It is a function that runs **automatically** when an object is created.
- In Python, the constructor method is named `__init__()`.
- It is used to assign **initial values** to object properties (like `name`, `age`, etc.).

Syntax of Constructor in Python

```
class ClassName:
```

```
def __init__(self, parameters):  
    # code to initialize the object
```

Example of Constructor

```
class Student:  
    def __init__(self, name, marks):    # constructor  
        self.name = name  
        self.marks = marks  
  
    def show(self):  
        print("Name:", self.name)  
        print("Marks:", self.marks)  
  
s1 = Student("Neeraj", 90)    # object created, constructor  
runs  
s1.show()
```

Output:

Name: Neeraj
Marks: 90

Types of Constructors in Python

Type	Description
Default Constructor	Constructor with no parameters
Parameterized Constructor	Constructor with parameters to initialize values

Default Constructor Example:

```
class Demo:  
    def __init__(self):    # No parameters  
        print("Object Created")
```

```
obj = Demo()
```

Parameterized Constructor Example:

```
class Demo:  
    def __init__(self, message):    # With parameter  
        print("Message:", message)
```

```
obj = Demo("Hello Python")
```

Summary

- `__init__()` is a **constructor**
- It runs **automatically** when an object is created
- Used to **initialize** object variables

Inheritance in python

Code **reusability**: You don't have to write the same code again.
It helps in **organizing** code in a hierarchical manner.
You can **extend** or **modify** behavior of the parent class in the child class.

Basic Syntax of Inheritance:

```
class Parent:
    def display(self):
        print("This is Parent class.")

class Child(Parent): # Inheriting Parent class
    def show(self):
        print("This is Child class.")

obj = Child()
obj.display() # Accessing Parent class method
obj.show()   # Accessing Child class method
```

Types of Inheritance in Python:

Type	Description	Example
Single	One child class inherits from one parent class	class B(A)
Multiple	One child inherits from more than one parent class	class C(A, B)
Multilevel	One class inherits from a child class which in turn inherits from another	class C(B), class B(A)
Hierarchical	Multiple child classes inherit from the same parent class	class B(A), class C(A)
Hybrid	Combination of more than one type of inheritance	Combination of above

Access Modifiers in Python

In Python, **access modifiers** are used to **control the visibility (accessibility)** of class members (variables and methods). Unlike some other languages like Java or C++, Python doesn't have strict access control, but it provides **naming conventions** to simulate them.

Types of Access Modifiers in Python

Modifier	Syntax Example	Access Level
Public	self.name	Accessible from anywhere
Protected	self._name	Suggests access within class and subclass
Private	self.__name	Not accessible outside the class (name mangled)

1. Public Members

Can be accessed **anywhere** (inside or outside the class).

```
class Student:
    def __init__(self):
        self.name = "Neeraj" # Public member

s = Student()
print(s.name) # Accessible
```

2. Protected Members

Defined with a **single underscore** `_`.

By convention, should not be accessed outside the class or its subclasses.

Python doesn't enforce this restriction.

```
class Student:
    def __init__(self):
        self._marks = 90 # Protected

class Derived(Student):
    def show(self):
        print("Marks:", self._marks)

d = Derived()
d.show()
print(d._marks) # Technically accessible, but not recommended
```

3. Private Members

Defined with **double underscore** `__`.

Python uses **name mangling** to make them harder to access from outside.

```
class Student:
    def __init__(self):
        self.__roll = 101 # Private

    def show(self):
        print("Roll No:", self.__roll)

s = Student()
s.show()
# print(s.__roll) # Error: 'Student' object has no attribute '__roll'
print(s._Student__roll) # Accessible via name mangling (not recommended)
```

Summary Table:

Modifier	Prefix	Access From Outside	Subclass Access	Notes
Public	none	Yes	Yes	Default access
Protected	<code>_var</code>	Yes (by convention no)	Yes	Just a naming convention
Private	<code>__var</code>	No (name mangled)	No (directly)	Still accessible via workaround

Tip:

Python focuses more on **developer discipline** than strict enforcement. Use private/protected when needed, but don't rely on them for security.